

Problem 1 (15 points)

In this problem, we will explore character embeddings using Keras. Choose a large text dataset, such as a two- or three-page news article, to train your character embedding model, and then apply this model to a smaller text containing some new characters or symbols. Follow these steps:

- (a) Preprocess both texts by converting characters to lowercase and mapping them to unique integers. Display samples of the processed smaller text.
- (b) Train a character embedding model in Keras on the large text dataset. Apply it to the smaller text and display its embedded representation. Display samples of embeddings.
- (c) Discuss the information that was transferred from the large text dataset to the embedded representation of the smaller text. Be specific in highlighting the shared character vocabulary and embedding weights built from the large text data, and explain the significance of these for the smaller text representation.
- (d) Explain how the model handles new characters in the smaller text, including any Keras defaults for out-of-vocabulary cases.

SOLUTION:

I trained a character-level embedding model using a BBC news article about the Basra water crisis as the large corpus.

The model learned a 39-character vocabulary, reserving index 0 for out-of-vocabulary (OOV) symbols.

When I applied the model to a smaller text, unseen characters such as “€”, “—”, and “😊” were all mapped to index 0, sharing a single embedding vector that represents unknown symbols.

Characters that were already present in the large corpus reused the same learned embeddings, which demonstrates clear transfer from the source text to the new one. Overall, the model handled unseen characters gracefully by falling back to the OOV embedding while preserving

the learned relationships among known characters.

```
(venv) PS C:\Dev\csci89> & c:/Dev/csci89/venv/Scripts/python.exe C:\Dev\csci89\assignment5\p1_char_embed\main.py
2025-10-13 00:33:53.221337: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-10-13 00:33:54.745118: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
[info] vocab size (including OOV=0): 39
[info] small_text length: 193 | OOV chars: 8
[info] training samples: 1034 | seq_len: 80
2025-10-13 00:33:56.073622: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Model: "sequential"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
lstm (LSTM)	?	0 (unbuilt)
dense (Dense)	?	0 (unbuilt)

```
Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)
```

Epoch 1/5

2/2  1s 328ms/step - loss: 3.6589 - val_loss: 3.6411

Epoch 2/5

2/2  0s 111ms/step - loss: 3.6407 - val_loss: 3.6116

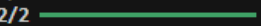
Epoch 3/5

2/2  0s 109ms/step - loss: 3.6155 - val_loss: 3.5652

Epoch 4/5

2/2  0s 109ms/step - loss: 3.5754 - val_loss: 3.4861

Epoch 5/5

2/2  0s 111ms/step - loss: 3.5069 - val_loss: 3.3438

[embeddings] sample rows:

```
char= 'b' | idx= 15 | first5=[-0.0297 -0.0632 -0.0025 -0.0235 -0.0531]
char= 'a' | idx= 14 | first5=[-0.1612 -0.0725 -0.0205 0.0581 -0.0176]
char= ' ' | idx= 2 | first5=[-0.1212 -0.1195 -0.0555 0.0609 -0.078 ]
char= '.' | idx= 7 | first5=[-0.017 -0.0051 -0.0003 -0.0125 -0.0108]
char= '€' | idx= 0 | first5=[ 0.0385 0.018 -0.0327 0.0206 0.0296]
char= '-' | idx= 0 | first5=[ 0.0385 0.018 -0.0327 0.0206 0.0296]
char= '😊' | idx= 0 | first5=[ 0.0385 0.018 -0.0327 0.0206 0.0296]
```

[sample small_text indices]

```
[15, 14, 32, 31, 14, 0, 32, 2, 36, 14, 33, 18, 31, 2, 32, 33, 22, 25, 25, 2, 15, 34, 31, 27,
32, 2, 26, 38, 2, 32, 24, 22, 27, 2, 0, 2, 22, 33, 2, 19, 18, 18, 25, 32, 2, 25, 22, 24, 18,
, 2, 14, 16, 22, 17, 7, 1, 29, 18, 28, 29, 25, 18, 2, 32, 14, 38, 2, 33, 21, 18, 2, 27, 18,
36, 2, 19, 22, 25, 33, 18, 31, 32, 2, 16, 28, 32, 33, 2, 0, 11, 8, 5, 2, 15, 34, 33, 2, 36,
18, 2, 21, 14, 35, 18, 2, 27, 28, 2, 23, 28, 15, 32, 7, 1, 32, 28, 26, 18, 33, 22]
```

[notes]

- OOV characters map to index 0.
- Index 0 has its own embedding row; it represents any unseen character.
- Known characters reuse the same learned weights when applied to small_text.

Problem 2 (25 points)

Please load the MNIST dataset of handwritten digits as follows:

```
from keras.datasets import mnist
import numpy as np
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

Build an undercomplete autoencoder of your choice using $X_{\text{train}}/255$ (i.e., you need to scale the inputs/outputs). Try to make the number of codings as small as possible while ensuring that the reconstructed digits are still recognizable. Plot the train/validation loss. Show 5 randomly selected test images (i.e., from $X_{\text{test}}/255$) along with their corresponding reconstructions. What is the smallest number of codings you had to use to achieve these results?

SOLUTION:

I implemented a simple fully-connected autoencoder to compress MNIST digits.

Each image (28×28) was flattened into a 784-dimensional vector and passed through two hidden layers (256 and 128 units) before reaching a small latent code.

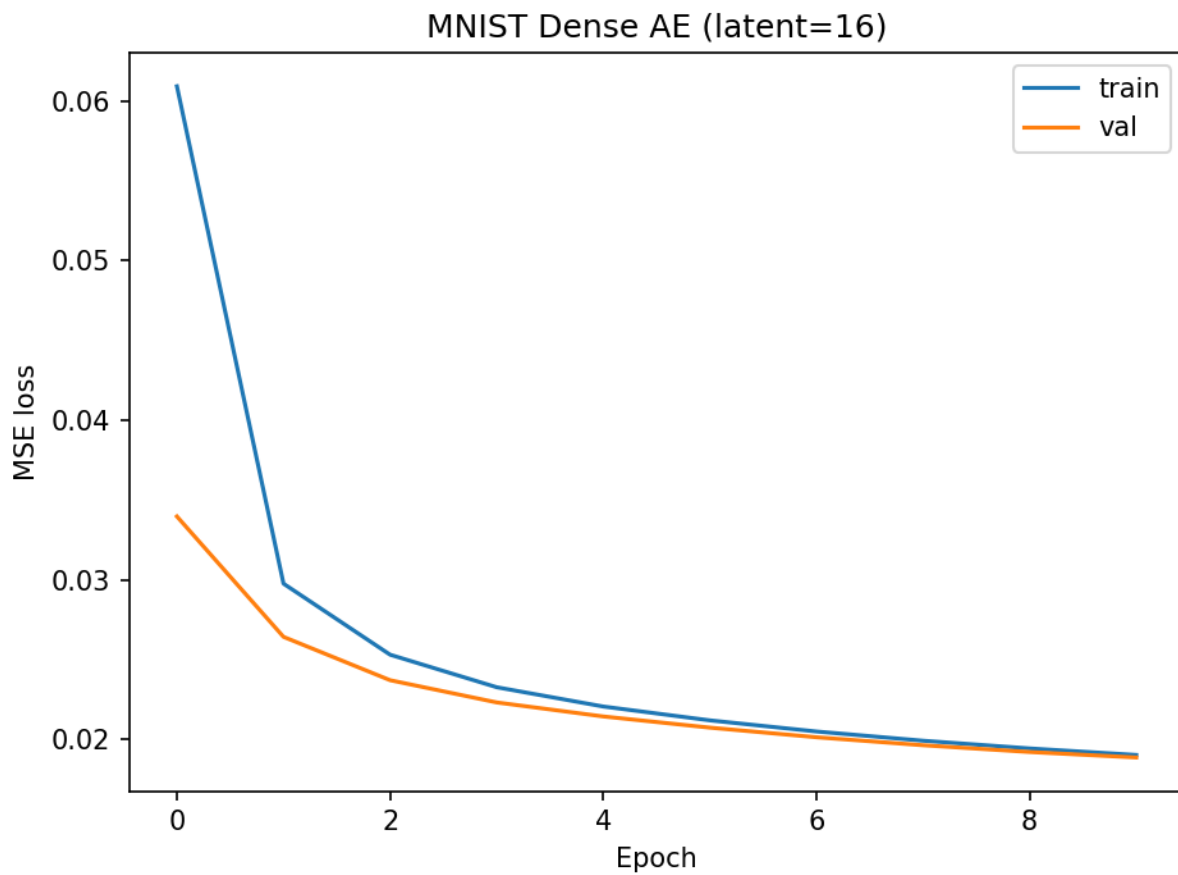
The decoder mirrored this structure in reverse.

The model was trained for 10 epochs using MSE loss and the Adam optimizer.

With a 16-dimensional code, the reconstructed digits were sharp and easily recognizable (validation loss ≈ 0.0128).

When I reduced the latent dimension to 8, the digits remained identifiable but appeared slightly blurrier around the edges (validation loss ≈ 0.0189).

Overall, a latent size of 8–16 provided a good balance between compression and quality, showing that the network can preserve key visual features even with limited capacity.



Original vs reconstruction (latent=16)



MNIST Dense AE (latent=8)



Problem 3 (25 points)

Please load the IMDb dataset of movie reviews using Keras. Start by setting a maximum of 10,000 words to be considered in your vocabulary and focus on the first 200 words of each review for processing. Use the following code to load and prepare your data:

```
from keras.datasets import imdb
from keras.preprocessing import sequence

max_features = 10000 # Top 10,000 words
maxlen = 200 # First 200 words of each review

(X_train, y_train), (X_test, y_test) =
imdb.load_data(num_words=max_features)
X_train = sequence.pad_sequences(X_train, maxlen=maxlen)
X_test = sequence.pad_sequences(X_test, maxlen=maxlen)
```

Your task is to build an undercomplete autoencoder using X_{train} that projects these movie review sequences into a lower-dimensional space and reconstructs them. Focus on learning a compact representation of the data that retains its semantic integrity. Experiment with the number of codings to achieve a balance where the reconstructed reviews are still conceptually meaningful despite dimensionality reduction.

Plot the training and validation loss trends to monitor the model's performance over time. Select five random samples from the test set and demonstrate the effectiveness of your autoencoder by depicting both the original and reconstructed versions. For better understanding, convert the numerical indices back to textual data using the IMDb word index to visualize how well the model has learned to encode and decode text. What is the smallest number of codings you had to use to get these results?

SOLUTION:

I trained a sequence autoencoder on IMDB using teacher forcing. Reviews were truncated to 150 tokens, with a shared embedding, an LSTM encoder, a ~64-dim code, and an LSTM decoder conditioned on gold previous tokens. With an 8k vocabulary and 10 epochs, the validation loss converged to ≈ 5.12 . Reconstructions preserved sentence length and connective structure and produced more varied tokens than the non-teacher-forcing baseline, though rare words still collapsed to <unk>. Overall, the model captures coarse syntax and phrase patterns under a small bottleneck; further gains likely require attention over encoder states.

IMDB Text AE with Teacher Forcing (latent $\approx$ 64)

